Decision Pipeline

Purpose

Decision Pipeline is the reasoning flow that transforms incoming input into character behavior.

Its purpose is not to generate language.

Its purpose is to decide **what should happen next**.

Every interaction with the outside world passes through the Decision Pipeline before affecting the character.

This ensures that every response, action and reaction follows a consistent architectural path.

Philosophy

Shio should never ask the language model to solve problems that can be solved deterministically.

Reasoning is expensive.

Rules are cheap.

The architecture therefore follows a layered decision model.

Each layer answers progressively more complex questions.

If a decision can be made earlier, later layers are never executed.

This minimizes latency, computational cost and unnecessary LLM usage.

Decision Flow

```text id="dp01"
External Input
    ▼
Intent Router
    ▼
Rule Gate
    ├──────────► Reject
    ▼
Optional Intent LLM
    ├──────────► Reject
    ▼
Main Character LLM
    ▼
Action Runtime
    ▼
Experience Layer
    ▼
Character State
```

```
Each layer exists to reduce uncertainty before reaching the next one.

---

# Layer 1 — Intent Router
```

```
Intent Router performs extremely lightweight classification.

Its responsibility is routing.

It never performs reasoning.

Typical classifications include:

- conversation;
- command;
- ignored speech;
- background speech;
- system request.

Intent Router should execute in milliseconds.

---

# Layer 2 — Rule Gate

Rule Gate contains deterministic rules.

Examples include:

- wake phrases;
- ignored filler words;
- duplicated recognition;
- confidence thresholds;
- cooldown timers;
- command prefixes.

If Rule Gate can confidently determine the outcome, the pipeline immediately
terminates.

No LLM should be involved.

---

# Layer 3 — Optional Intent LLM

Some situations cannot be expressed using deterministic rules.

Examples include:

- "Was the character actually addressed?"
- "Is this part of the current conversation?"
- "Is this rhetorical?"
- "Should Shio politely ignore this?"

Only these ambiguous situations reach the Intent LLM.
```

````
Its output is intentionally simple.

Examples:

```text id="dp02"
RESPOND

IGNORE

SYSTEM_COMMAND

TOOL_REQUEST
````

The Intent LLM never generates dialogue.

It never calls tools.

It never modifies Character State.

It only reduces uncertainty.

Layer 4 — Character LLM

Only after the previous layers approve the interaction does the request reach the primary language model.

The Character LLM is responsible for:

- personality;
- dialogue;
- humor;
- emotional tone;
- creativity;
- reasoning;
- tool intentions.

The Character LLM never directly performs actions.

It proposes them.

Execution belongs elsewhere.

Layer 5 — Action Runtime

If the Character LLM requests an action, execution is delegated to Action Runtime.
````

Examples include:

- moderating chat;
- changing application settings;
- interacting with external services;
- future integrations.

Action Runtime validates every request before execution.

The Character LLM never bypasses Runtime validation.

Layer 6 — Experience Layer

Execution does not conclude the interaction.

Completed actions become experiences.

Experience Layer determines how the character emotionally reacts.

Examples include:

- making a joke;
- smiling;
- remaining silent;
- expressing disappointment;
- acting proud after success.

Execution changes the world.

Experience changes the character.

Why This Pipeline Exists

Separating the decision process into multiple layers provides several benefits.

Lower Latency

Simple decisions never require expensive reasoning.

Lower Cost

Many interactions terminate before reaching the primary language model.

Better Safety

Potentially dangerous actions pass through deterministic Runtime validation.

Better Personality

The Character LLM focuses exclusively on behaving like Shio instead of solving routing or system problems.

Better Maintainability

Each decision layer has one clearly defined responsibility.

No layer becomes excessively complex.

Architectural Guarantees

The Decision Pipeline guarantees the following:

- deterministic decisions are preferred over probabilistic reasoning;
- small models never replace the primary character model;
- the Character LLM remains focused on personality rather than infrastructure;
- Runtime always owns execution;
- every completed action may become an experience;
- no single component becomes responsible for the entire decision process.

These guarantees preserve both architectural clarity and the illusion of interacting with a coherent digital character.

Future Evolution

The Decision Pipeline is expected to evolve incrementally.

Future additions may include:

- passive companion routing;
- scheduler-triggered interactions;
- memory-driven decisions;
- multi-agent coordination;
- contextual priority management.

New layers should extend the pipeline rather than replace existing responsibilities.

The overall architectural philosophy remains unchanged:

The simplest component capable of making the correct decision should always make that decision.